

Estruturas de Dados

Processamento de Expressões Matemáticas

Material completo de estudo e apresentação — Trabalho 02

Pilha encadeada · Notação pós-fixa · Algoritmo de conversão

Integrante	Matrícula
Rafael Peixoto Gonçalves	2025010288
Antonio Everardo Liveira Lima Filho	2025010334
Cid Xavier Pacheco Araujo	2025010285

Disciplina: Estruturas de Dados — Sistemas de Informação · Prof. Raul de Araújo Lima

Centro Universitário Católica de Quixadá (Unicatólica) · Python 3 (apenas biblioteca padrão)

Entrega: 02/10/2026 · Apresentação: 05/10/2026

Sumário

1. Visão geral do trabalho	3
2. A estrutura de dados: Pilha encadeada	5
3. Desafio 1: avaliar expressão pós-fixa	8
4. Desafio 2: converter para pós-fixa	12
5. O programa principal: main.py	20
6. Testes (resultados reais do código)	22
7. Decisões de implementação e limitações	23
8. Modificações que o professor pode pedir (com solução testada)	24
9. Perguntas prováveis e respostas	26
10. Roteiro sugerido para a apresentação	28
11. Declaração de uso de ferramentas de IA	28

1. Visão geral do trabalho

Quando digitamos $3 + 4 * 2$ em uma calculadora, alguma coisa precisa transformar essa sequência de caracteres em um número. O trabalho consiste em construir esse "alguma coisa": um programa que recebe uma expressão matemática escrita da forma usual (com parênteses e precedência de operadores), calcula o resultado e **mostra, passo a passo, o que acontece dentro das pilhas** usadas no processo.

1.1. O problema: ler um operador não é poder aplicá-lo

Se lermos a expressão da esquerda para a direita, um símbolo por vez, e tentarmos calcular assim que possível, erramos:

```
3    guardo o 3
+    ...e agora? não sei com o que somar
4    seria 3 + 4? ou vem algo depois?
*    de fato vinha: o + não é 3 + 4, mas sim 3 + 8 (porque 4 * 2 vem antes)
2
```

Na notação que usamos no dia a dia (chamada **infixa**, pois o operador fica *entre* os operandos), o que vem depois de um operador pode mudar a quem ele se aplica. A precedência e os parênteses só fazem sentido quando a expressão inteira já foi lida.

1.2. Por que uma pilha resolve isso

Se não podemos aplicar o operador agora, precisamos **guardá-lo e retomá-lo depois**. A pergunta seguinte é: retomar em que ordem? Em $3 + 4 * 2$, o $*$ foi escrito depois do $+$, mas precisa ser resolvido antes dele. Em $(3 + 4) * 2$, o parêntese inverte essa ordem. O que se repete nos dois casos é que a decisão envolve sempre o item **mais recente** que ficou pendente.

Ideia central do trabalho

Guardar itens e retomar o **mais recente primeiro** é exatamente o contrato LIFO (*Last In, First Out*) da pilha. A pilha não está aqui por conveniência de implementação: a própria **estrutura do problema** é a de uma pilha.

1.3. A solução em duas etapas

O problema é dividido em dois desafios independentes, cada um com sua própria pilha. O programa final encadeia os dois: a saída do Desafio 2 é exatamente a entrada que o Desafio 1 consome.

Desafio	Entrada	Saída	O que a pilha guarda
1. Avaliar expressão pós-fixa	3 4 2 * +	11	Números (operandos) que estão esperando um operador
2. Converter para pós-fixa	3 + 4 * 2	3 4 2 * +	Operadores e parênteses que ainda não puderam ser aplicados/enviados à saída

Fluxo completo do programa para a entrada **3 + 4 * 2**:

```
"3 + 4 * 2"                                (string digitada)
  |
  | tokenizar()  ->  ['3','+', '4','*', '2']
  v
converter()  (Desafio 2, pilha de OPERADORES)
  |
  v
"3 4 2 * +"                                (pós-fixa)
  |
  | tokenizar() do Desafio 1  ->  ['3','4','2','*', '+']
  v
avaliar()    (Desafio 1, pilha de NÚMEROS)
  |
  v
11                                                  (resultado)
```

1.4. Organização dos arquivos

Arquivo	Conteúdo
desafio1.py	Classes No e Pilha; funções fmt, tokenizar (pós-fixa) e avaliar
desafio2.py	Constantes PRIORIDADE e DIREITA; funções tokenizar (notação usual) e converter
main.py	Programa principal: funções de impressão, processar e main (lê, converte, avalia, imprime)
registro.md	Readme do projeto, com integrantes, explicação do código e declaração de uso de IA

Regra de ouro da arquitetura

As funções **tokenizar**, **converter** e **avaliar** **não imprimem nada**. Elas devolvem dados (o rastreamento, o resultado e o erro). Toda a impressão fica na **main.py**. Isso é uma exigência do enunciado e uma boa prática: separar a lógica da apresentação.

2. A estrutura de dados: Pilha encadeada

A pilha foi implementada de forma **encadeada**, sem usar *list* do Python internamente. Em vez de um vetor, a pilha é uma corrente de pequenos objetos (nós), em que cada nó guarda um valor e aponta para o nó que está logo abaixo dele.

2.1. A classe No

```
class No:
    def __init__(self, valor, proximo=None):
        self.valor = valor
        self.proximo = proximo
```

- **valor**: o dado armazenado (um número no Desafio 1; um operador, parêntese ou token no Desafio 2).
- **proximo**: referência ao nó imediatamente abaixo. Vale None no nó da base, que é o fim da corrente.
- `proximo=None` como padrão permite criar um nó sem nada abaixo (primeiro elemento da pilha).

2.2. A classe Pilha: atributos

```
def __init__(self):
    self.topo_no = None
    self.contador = 0
```

- **topo_no**: referência ao nó do topo. Pilha vazia significa `topo_no is None`. Só o topo é acessado diretamente; os demais nós são alcançados seguindo `proximo`.
- **contador**: quantos elementos há na pilha. Existe para que `tamanho()` responda sem percorrer toda a corrente.

Veja como a pilha fica depois de empilhar 3, depois 4, depois 2:

```
topo_no
|
v
+-----+-----+      +-----+-----+      +-----+-----+
| valor | proximo|      | valor | proximo|      | valor | proximo|
|  2   |  ----->|  4   |  ----->|  3   |  None  |
+-----+-----+      +-----+-----+      +-----+-----+
      (topo)                                (base)

contador = 3
```

2.3. Os métodos do contrato, um por um

empilhar(novo_valor)

```
def empilhar(self, novo_valor):
    self.topo_no = No(novo_valor, self.topo_no)
    self.contador += 1
```

Cria um nó novo cujo `proximo` é o topo *atual* e, em seguida, faz esse nó novo ser o topo. Na prática, o nó novo "se pendura" na frente da corrente. Por fim, incrementa o contador. São poucas operações fixas, independentes do tamanho da pilha: custo **O(1)**.

desempilhar()

```
def desempilhar(self):
    valor_removido = self.topo_no.valor
    self.topo_no = self.topo_no.proximo
    self.contador -= 1
    return valor_removido
```

- Linha 1: guarda o valor do topo em uma variável, pois o nó será descartado.
- Linha 2: o topo passa a ser o nó de baixo (`topo_no.proximo`). O antigo topo deixa de ser referenciado e o Python o descarta.
- Linha 3: decrementa o contador. Linha 4: devolve o valor removido.
- **O(1)**. Atenção: o método **não verifica se a pilha está vazia**; chamar em pilha vazia causaria erro (`None.valor`). Quem usa é responsável por checar antes. O *avaliar* confere `tamanho() < 2` e o *converter* confere `vazia()`.

topo(), vazia() e tamanho()

```
def topo(self):
    return self.topo_no.valor

def vazia(self):
    return self.topo_no is None

def tamanho(self):
    return self.contador
```

- **topo()**: espia o valor do topo **sem remover**. É a operação de "olhar o item mais recente" que o algoritmo de conversão usa o tempo todo.
- **vazia()**: devolve True se não há nó algum.
- **tamanho()**: devolve o contador. Sem ele, seria preciso percorrer a corrente ($O(n)$).

2.4. O método extra: visualizar()

```
def visualizar(self):
    # copia os valores da base até o topo, sem mexer na pilha
    itens = []
    atual = self.topo_no
    while atual is not None:
        itens.append(atual.valor)
        atual = atual.proximo
    itens.reverse()
    return itens
```

O enunciado exige que o rastreamento mostre o conteúdo da pilha a cada passo **sem destruí-la**. Como a pilha é encadeada, não dá para simplesmente imprimi-la: é preciso percorrer os nós. O método faz isso assim:

- Cria uma lista vazia `itens` e uma variável auxiliar `atual` que começa no topo.
- Enquanto `atual` não for None, copia o valor para a lista e avança para o nó de baixo. O `topo_no` **nunca é alterado**, por isso a pilha original fica intacta.
- Como a corrente foi percorrida do topo para a base, a lista está "de cabeça para baixo". O `reverse()` a coloca da **base para o topo**, que é o formato pedido.
- Devolve uma **cópia** em forma de lista: mexer nela não afeta a pilha.

Custo **O(n)**, pois percorre todos os nós; é usado apenas para imprimir o rastreamento.

2.5. Resumo de complexidade

Método	Custo	Por quê
empilhar	$O(1)$	cria um nó e troca o topo
desempilhar	$O(1)$	só avança o topo uma posição
topo / vazia / tamanho	$O(1)$	só leem topo_no ou contador
visualizar	$O(n)$	percorre os n nós uma vez

3. Desafio 1: avaliar expressão pós-fixa

3.1. O que é a notação pós-fixa

Na notação pós-fixa (ou polonesa reversa), o operador vem **depois** dos seus operandos. Repare no que desaparece: **não há parênteses e não há regra de precedência**. A própria ordem dos símbolos já resolveu toda a ambiguidade, e por isso é a notação usada internamente por calculadoras e máquinas virtuais.

Notação usual	Pós-fixa
$3 + 4$	$3\ 4\ +$
$3 + 4 * 2$	$3\ 4\ 2\ *\ +$
$(3 + 4) * 2$	$3\ 4\ +\ 2\ *$

3.2. A ideia do algoritmo

Lê-se a expressão da esquerda para a direita, um token por vez:

- Se o token é um **número**, guarda-se na pilha (ele ainda não pode ser usado: falta o operador).
- Se o token é um **operador**, ele se aplica aos **dois números mais recentes**, que são justamente os dois do topo da pilha. Tiram-se os dois, calcula-se e empilha-se o resultado.
- No fim, deve sobrar exatamente **um** número na pilha: o resultado.

O erro mais comum

Ordem dos operandos. O primeiro a sair da pilha (o do topo) é o operando da **direita**; o segundo a sair é o da **esquerda**. Por isso o código faz `b = desempilhar()` e *depois* `a = desempilhar()`, e calcula `a op b`. Em **5 2 -** o resultado é $5 - 2 = 3$ (e não $2 - 5$). Expressões só com `+` e `*` escondem esse erro porque são comutativas, e por isso o enunciado manda testar com `-` e `/` desde cedo.

3.3. Funções auxiliares: fmt e tokenizar

```
def fmt(v):  
    """Imprime 8 em vez de 8.0, mas mantém 2.5 como 2.5."""  
    return f"{v:g}"
```

```
def tokenizar(expressao):  
    return expressao.split()
```

- **fmt(v)**: o formato `:g` remove zeros desnecessários: `8.0` vira `"8"` e `2.5` continua `"2.5"`. Todos os valores impressos passam por ela. Serve só para exibição, não para o cálculo.
- **tokenizar(expressao)** (Desafio 1): na pós-fixa os tokens já vêm separados por espaço, então `split()` basta.

3.4. A função avaliar, parte por parte

```

1 | def avaliar(tokens):
2 |     pilha = Pilha()
3 |     trace = []
4 |
5 |     for token in tokens:
6 |         if token in ("+", "-", "*", "/", "^"):
7 |             if pilha.tamanho() < 2:
8 |                 return trace, None, "erro: operandos insuficientes"
9 |
10 |             # b sai primeiro, depois a
11 |             b = pilha.desempilhar()
12 |             a = pilha.desempilhar()
13 |
14 |             if token == "+":
15 |                 resultado = a + b
16 |             elif token == "-":
17 |                 resultado = a - b
18 |             elif token == "*":
19 |                 resultado = a * b
20 |             elif token == "/":
21 |                 if b == 0:
22 |                     return trace, None, "erro: divisao por zero"
23 |                 resultado = a / b
24 |             elif token == "^":
25 |                 try:
26 |                     resultado = a ** b
27 |                 except (OverflowError, ZeroDivisionError):
28 |                     return trace, None, "erro: resultado invalido"
29 |                 if isinstance(resultado, complex):
30 |                     return trace, None, "erro: resultado invalido"
31 |
32 |             pilha.empilhar(resultado)
33 |         else:
34 |             try:
35 |                 pilha.empilhar(float(token))
36 |             except ValueError:
37 |                 return trace, None, "erro: caractere invalido"
38 |
39 |         trace.append((token, pilha.visualizar()))
40 |
41 |     if pilha.tamanho() != 1:
42 |         return trace, None, "erro: expressao malformada"
43 |
44 |     return trace, pilha.topo(), None

```

A função devolve sempre uma tripla (trace, resultado, erro): o rastreamento acumulado, o valor (ou None) e a mensagem de erro (ou None). Explicação por trechos:

Linhas	O que faz
2-3	Cria a pilha de números e a lista <i>trace</i> , que guardará uma "foto" da pilha após cada token.
5-6	Percorre os tokens. Se o token é um dos cinco operadores, entra no ramo do operador.
7-8	Validação: um operador precisa de 2 operandos. Se a pilha tem menos de 2, devolve o erro "operandos insuficientes" (junto com o trace feito até aqui).
10-12	Desempilha b (topo = direita) e depois a (esquerda).
14-19	+, - e *: calcula direto com <i>a op b</i> .
20-23	Divisão: se <i>b</i> é zero, devolve "divisao por zero" antes de dividir. Senão, <i>a / b</i> .

Linhas	O que faz
24-30	Potência: $a ** b$ pode estourar (OverflowError), dar divisão por zero (0 elevado a negativo) ou resultar em número complexo (ex.: -8 elevado a 0.5). Os três casos viram "erro: resultado invalido", sem exceção não tratada.
32	Empilha o resultado da operação.
33-37	Ramo do número: tenta <code>float(token)</code> e empilha. Se o token não é número (ValueError), devolve "caractere invalido".
39	Depois de cada token processado, guarda (token, cópia da pilha). Como <code>visualizar()</code> devolve cópia, as fotos não se misturam.
41-42	Fim: se a pilha não tem exatamente 1 elemento, sobraram operandos (ex.: 3 4 5 +) e é "expressao malformada".
44	Tudo certo: devolve o trace, o valor do topo e erro = None.

3.5. Rastreamento passo a passo (saída real do programa)

Para a pós-fixa **3 4 2 * +** (que é $3 + 4 * 2$). Observe o ***** consumindo o 4 e o 2 e deixando o 8, e o **+** consumindo 3 e 8:

```
Entrada: 3 4 2 * +
token| pilha
3    | 3
4    | 3 4
2    | 3 4 2
*    | 3 8
+    | 11
resultado: 11
```

Com subtração, vendo a ordem b-depois-a funcionar (**10 3 -** dá $10 - 3 = 7$):

```
Entrada: 10 3 - 2 -
token| pilha
10   | 10
3    | 10 3
-    | 7
2    | 7 2
-    | 5
resultado: 5
```

3.6. Expressões inválidas

Situação	Exemplo	Mensagem	Por que acontece
Operandos insuficientes	$3 + * 4$	erro: operandos insuficientes	O + chega com só 1 número na pilha
Operandos sobrando	$3 4 5 +$	erro: expressao malformada	Sobram 2 números na pilha ao final
Divisão por zero	$8 0 /$	erro: divisao por zero	$b == 0$ é testado antes da divisão
Token inválido	$3 x +$	erro: caractere invalido	<code>float("x")</code> levanta ValueError
Potência inválida	$-8 0.5 ^$	erro: resultado invalido	Resultado seria complexo

Exemplos reais de erro, com o rastreamento até o ponto da falha:

```
Entrada: 3 4 5 +
token| pilha
3    | 3
4    | 3 4
5    | 3 4 5
+    | 3 9
erro: expressao malformada
```

```
Entrada: 8 0 /
token| pilha
8    | 8
0    | 8 0
erro: divisao por zero
```

4. Desafio 2: converter para pós-fixa

Agora a entrada é a notação usual, com parênteses e precedência, como em $3 + 4 * (2 - 1)$. Esta é a etapa em que a ambiguidade da notação precisa ser resolvida, e é aqui que a pilha faz o trabalho pesado. O algoritmo usado é o clássico **shunting-yard** (pátio de manobras), de Dijkstra.

4.1. Regras da notação

Operador	Precedência	Associatividade
+ -	1 (mais fraca)	esquerda
* /	2	esquerda
^	3 (mais forte)	direita

- **Associar à esquerda:** $10 - 3 - 2$ equivale a $(10 - 3) - 2 = 5$.
- **Associar à direita:** $2 ^ 3 ^ 2$ equivale a $2 ^ (3 ^ 2) = 2 ^ 9 = 512$ (e não 64).
- Os parênteses têm prioridade sobre tudo.

No código, essas regras viram dados (e não uma cadeia de if/else):

```
PRIORIDADE = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}
DIREITA = {"^"} # operadores resolvidos da direita para a esquerda
```

- **PRIORIDADE:** dicionário que associa cada operador ao seu nível. Quanto maior o número, mais forte.
- **DIREITA:** conjunto dos operadores associativos à direita. Só o $^$. Todo operador que *não* está aqui é associativo à esquerda.
- Vantagem: para adicionar um operador novo, basta mexer nessas tabelas.

4.2. Tokenização: separar a string em pedaços

A entrada pode vir **sem espaços**, como em $3+42*(7-1)$. Aqui o `split()` não funciona (não há espaços para separar), e uma pilha também não ajudaria. O que resolve é perceber **o que distingue um caractere que continua o número anterior de um que inicia um token novo**: dígitos e ponto continuam o número; qualquer outro símbolo começa um token novo.

```

1 | def tokenizar(expressao):
2 |     tokens = []
3 |     inicio = 0
4 |     tamanho = len(expressao)
5 |
6 |     while inicio < tamanho:
7 |         caractere = expressao[inicio]
8 |
9 |         # espaço: ignora
10 |        if caractere.isspace():
11 |            inicio += 1
12 |
13 |        # número: pode ter vários dígitos (12, 2.5)
14 |        elif caractere.isdigit() or caractere == ".":
15 |            fim = inicio
16 |            while fim < tamanho and (expressao[fim].isdigit() or expressao[fim] == "."):
17 |                fim += 1
18 |
19 |            numero = expressao[inicio:fim]
20 |            try:
21 |                float(numero) # recusa "1.2.3" e "."
22 |            except ValueError:
23 |                return None, "erro: caractere invalido"
24 |
25 |            tokens.append(numero)
26 |            inicio = fim
27 |
28 |        # operador ou parêntese
29 |        elif caractere in "+-*/^()":
30 |            tokens.append(caractere)
31 |            inicio += 1
32 |
33 |        else:
34 |            return None, "erro: caractere invalido"
35 |
36 |    return tokens, None

```

Linhas	O que faz
2-4	<i>tokens</i> : lista de saída. <i>inicio</i> : posição do caractere que está sendo lido. <i>tamanho</i> : comprimento da string.
6-7	Laço principal: avança pela string. Pega o caractere atual.
10-11	Espaço: ignora e avança 1.
14-15	Dígito ou ponto: é o início de um número. Usa um segundo índice, <i>fim</i> , que parte de <i>inicio</i> .
16-17	<i>fim</i> avança enquanto o caractere ainda for dígito ou ponto. Ao parar, o número ocupa o trecho de <i>inicio</i> até <i>fim</i> (exclusivo).
19-23	Recorta o trecho <code>expressao[inicio:fim]</code> e valida com <code>float()</code> . Isso recusa "1.2.3" e "." sozinho (erro: caractere invalido).
25-26	Guarda o número como UM token e move <i>inicio</i> para <i>fim</i> (pula o número inteiro).
29-31	Operador ou parêntese: cada símbolo é um token de 1 caractere.
33-34	Qualquer outro caractere (ex.: \$, letras): "erro: caractere invalido".
36	Devolve (<i>tokens</i> , <i>None</i>) em caso de sucesso. Em erro, devolve (<i>None</i> , mensagem).

Resultado para $3+42*(7-1)$:

```
['3', '+', '42', '*', '(', '7', '-', '1', ')']
```

4.3. A função converter: o coração do trabalho

Recebe a **string**, chama o *tokenizar* e devolve (*trace*, *posfixa*, *erro*), em que *posfixa* é uma string como "3 4 2 * +". Usa duas coisas: a **pilha de operadores** (objeto Pilha) e a lista **saida**, onde a pós-fixa vai sendo montada.

```

1 | def converter(expressao):
2 |     # recebe a string e devolve: trace, posfixa (string), erro
3 |     tokens, erro = tokenizar(expressao)
4 |     if erro:
5 |         return [], None, erro
6 |
7 |     operadores = Pilha()
8 |     saida = []
9 |     trace = []
10 |
11 |     for token in tokens:
12 |
13 |         # abre parêntese: só empilha
14 |         if token == "(":
15 |             operadores.empilhar(token)
16 |
17 |         # fecha parêntese: despeja operadores até achar o "("
18 |         elif token == ")":
19 |             while not operadores.vazia() and operadores.topo() != "(":
20 |                 saida.append(operadores.desempilhar())
21 |
22 |             if operadores.vazia():
23 |                 return trace, None, "erro: parentese fechado sem abertura"
24 |
25 |             operadores.desempilhar() # descarta o "("
26 |
27 |         # operador: tira da pilha quem deve sair antes
28 |         elif token in PRIORIDADE:
29 |             while not operadores.vazia() and operadores.topo() != "(":
30 |                 operador_topo = operadores.topo()
31 |
32 |                 if token in DIREITA:
33 |                     # ^ : o topo só sai se for estritamente mais forte
34 |                     if PRIORIDADE[operador_topo] > PRIORIDADE[token]:
35 |                         saida.append(operadores.desempilhar())
36 |                     else:
37 |                         break
38 |                 else:
39 |                     # outros: o topo sai se for mais forte ou igual
40 |                     if PRIORIDADE[operador_topo] >= PRIORIDADE[token]:
41 |                         saida.append(operadores.desempilhar())
42 |                     else:
43 |                         break
44 |
45 |                 operadores.empilhar(token)
46 |
47 |         # número: vai direto para a saída
48 |         else:
49 |             saida.append(token)
50 |
51 |         # foto do momento (list(saida) é uma cópia)
52 |         trace.append((token, operadores.visualizar(), list(saida)))
53 |
54 |     # fim: despeja o que sobrou na pilha
55 |     while not operadores.vazia():
56 |         if operadores.topo() == "(":
57 |             return trace, None, "erro: parentese aberto sem fechamento"
58 |             saida.append(operadores.desempilhar())
59 |
60 |     trace.append(("fim", operadores.visualizar(), list(saida)))
61 |
62 |     return trace, " ".join(saida), None

```

Mapa das linhas da função:

Linhas	O que faz
3-5	Tokeniza a string; se houve caractere inválido, propaga o erro (com trace vazio).
7-9	Cria a pilha de operadores, a lista de saída e a lista do trace.
11	Laço principal: um token por vez.
14-15	Ramo do (: só empilha.
18-25	Ramo do) : despeja até o (, detecta) sem abertura (linhas 22-23) e descarta o ((linha 25).
28-45	Ramo do operador: laço que decide quem sai (30-43) e, por fim, empilha o operador atual (45).
48-49	Ramo do número: vai direto para a saída.
52	Foto do momento: token, pilha de operadores e cópia da saída.
55-58	Fim da leitura: despeja o que sobrou; se aparecer um (aqui, ele nunca foi fechado (linhas 56-57).
60	Última linha do trace, com o token literal (fim).
62	Devolve o trace, a pós-fixa como string (juntando a saída com espaços) e erro = None.

Cada tipo de token tem sua regra:

Token	Ação	Por quê
Número	Vai direto para a saída	Na pós-fixa, a ordem dos números não muda em relação à original
(	Empilha	Marca o início de um grupo. Funciona como "muro": nada abaixo dele sai enquanto não for fechado
)	Despeja operadores da pilha na saída até achar o (, e então descarta o (	Tudo que estava dentro do parêntese precisa ser resolvido antes. Se a pilha esvaziar sem achar o (, há) sobrando
Operador	Enquanto o topo "precisa sair antes", move o topo para a saída. Depois empilha o operador atual	Operadores mais fortes (ou de igual força, conforme associatividade) já pendentes devem ser resolvidos primeiro
(fim)	Move tudo o que restou na pilha para a saída	Não há mais nada a ler; os pendentes são aplicados do mais recente para o mais antigo

4.4. A decisão central: quem sai da pilha quando chega um operador?

Chega um operador *token* e há um operador *topo* na pilha (pendente). Precisamos decidir: o *topo* deve ser resolvido **antes** do *token*? Se sim, ele sai da pilha para a saída. O código do ramo de operador é:

```
while not operadores.vazia() and operadores.topo() != "(":
    operador_topo = operadores.topo()
    if token in DIREITA:
        # ^ : o topo só sai se for estritamente mais forte
        if PRIORIDADE[operador_topo] > PRIORIDADE[token]:
            saida.append(operadores.desempilhar())
        else:
            break
    else:
        # outros: o topo sai se for mais forte ou igual
        if PRIORIDADE[operador_topo] >= PRIORIDADE[token]:
            saida.append(operadores.desempilhar())
        else:
            break
operadores.empilhar(token)
```

- **topo() != "("**: o parêntese de abertura nunca sai por este caminho; ele é o "muro" e só é removido pelo ramo do).
- **Associativo à esquerda (+ - * /)**: o topo sai se tiver precedência **maior ou igual**. O "igual" é o que faz **10 - 3 - 2** virar **10 3 - 2** -: quando chega o segundo -, o primeiro (igual) sai antes, então resolve da esquerda para a direita.
- **Associativo à direita (^)**: o topo só sai se for **estritamente maior**. Com *igual*, o topo fica na pilha: quando chega o segundo ^ em **2 ^ 3 ^ 2**, o primeiro ^ **não sai**, então o último ^ é resolvido primeiro: **2 3 2 ^ ^**.
- O **break** interrompe o laço assim que o topo não precisa sair: tudo abaixo dele é ainda mais antigo e também não sairá.

Por que dois critérios?

Só a **combinação correta** acerta os três casos de teste de associatividade. Tratar tudo como esquerda dá 64 em **2 ^ 3 ^ 2**; tratar tudo como direita dá 9 em **10 - 3 - 2**. É exatamente por isso que o código tem os dois critérios ($\geq$ e $>$).

4.5. Rastreamentos reais (saída do programa)

Exemplo 1: precedência ($3 + 4 * 2$). Observe o * ficar **empilhado em cima do +** (pilha "+ *"): ele será resolvido antes. Na saída final, o * aparece antes do +:

token	pilha	saida
3	-	3
+	+	3
4	+	3 4
*	+ *	3 4
2	+ *	3 4 2
(fim)	-	3 4 2 * +

posfixa: 3 4 2 * +

token	pilha
3	3
4	3 4
2	3 4 2
*	3 8
+	11

resultado: 11

Exemplo 2: parênteses $(3 + 4 * (2 - 1))$. O (entra na pilha como muro; o - fica em cima dele; o) despeja o - e descarta o (:

token	pilha	saida
3	-	3
+	+	3
4	+	3 4
*	+ *	3 4
(	+ * (	3 4
2	+ * (	3 4 2
-	+ * (-	3 4 2
1	+ * (-	3 4 2 1
)	+ *	3 4 2 1 -
(fim)	-	3 4 2 1 - * +

posfixa: 3 4 2 1 - * +

token	pilha
3	3
4	3 4
2	3 4 2
1	3 4 2 1
-	3 4 1
*	3 4
+	7

resultado: 7

Exemplo 3: associatividade à esquerda $(10 - 3 - 2)$. Quando o segundo - chega, o primeiro (de mesma precedência) sai antes para a saída:

token	pilha	saida
10	-	10
-	-	10
3	-	10 3
-	-	10 3 -
2	-	10 3 - 2
(fim)	-	10 3 - 2 -

posfixa: 10 3 - 2 -

token	pilha
10	10
3	10 3
-	7
2	7 2
-	5

resultado: 5

Exemplo 4: associatividade à direita $(2 \wedge 3 \wedge 2)$. A pilha acumula " $\wedge \wedge$ ": o segundo $\wedge$ não expulsa o primeiro:

token	pilha	saida
2	-	2
$\wedge$	$\wedge$	2
3	$\wedge$	2 3
$\wedge$	$\wedge \wedge$	2 3
2	$\wedge \wedge$	2 3 2
(fim)	-	2 3 2 $\wedge \wedge$

posfixa: 2 3 2 $\wedge \wedge$

token	pilha
2	2
3	2 3
2	2 3 2
$\wedge$	2 9
$\wedge$	512

resultado: 512

4.6. Expressões inválidas

Situação	Exemplo	Mensagem	Onde é detectado
Fechamento sem abertura	3 + 4)	erro: parentese fechado sem abertura	No ramo do): a pilha esvazia sem achar o (
Abertura sem fechamento	(3 + 4	erro: parentese aberto sem fechamento	No esvaziamento final: sobrou um (na pilha
Caractere não reconhecido	3 \$ 4	erro: caractere invalido	No tokenizar

```
Entrada: 3 + 4)
token| pilha      | saida
3    | -              | 3
+    | +              | 3
4    | +              | 3 4
erro: parentese fechado sem abertura
```

```
Entrada: (3 + 4
token| pilha      | saida
(    | (            | -
3    | (            | 3
+    | ( +          | 3
4    | ( +          | 3 4
erro: parentese aberto sem fechamento
```

```
Entrada: 3 $ 4
erro: caractere invalido
```

5. O programa principal: main.py

A **main.py** é a única que imprime. Ela junta as peças: pede a expressão, converte (Desafio 2), imprime o rastreamento, avalia (Desafio 1), imprime o rastreamento e mostra o resultado ou o erro.

5.1. Importações

```
from desafio1 import fmt, avaliar, tokenizar as tokenizar_posfixa
from desafio2 import converter
```

Os dois arquivos têm uma função chamada `tokenizar` (a do Desafio 1 só faz *split*; a do Desafio 2 lê caractere a caractere). Para não haver conflito de nomes, a do Desafio 1 é importada com o apelido **`tokenizar_posfixa`**. O `converter` chama a sua própria `tokenizar` internamente, por isso a `main` só precisa da outra, para separar a pós-fixa que o `converter` devolve.

5.2. Funções de impressão

```
def texto(lista):
    # lista vazia vira "-"
    if not lista:
        return "-"
    return " ".join(lista)

def imprimir_desafio2(trace):
    if not trace:
        return
    print(f"{'token':<5}| {'pilha':<11}| saída")
    for token, pilha, saida in trace:
        print(f"{'token':<5}| {texto(pilha):<11}| {texto(saida)}")

def imprimir_desafio1(trace):
    if not trace:
        return
    print(f"{'token':<5}| pilha")
    for token, pilha in trace:
        numeros = " ".join(fmt(v) for v in pilha)
        print(f"{'token':<5}| {numeros}")
```

- **`texto(lista)`**: junta os itens com espaço. Lista vazia vira "-", como o enunciado manda para pilha ou saída vazias.
- **`imprimir_desafio2`**: cabeçalho e uma linha por foto do `trace`. `f"{'token':<5}"` significa "alinhar à esquerda em 5 colunas"; `{texto(pilha):<11}` faz o mesmo com 11 colunas para a pilha. O formato pedido é token em 5, pilha em 11, separados por "|".
- **`imprimir_desafio1`**: igual, porém só com token e pilha. Cada valor passa por **`fmt`** (8.0 vira 8).
- Ambas começam com `if not trace: return`: se houve erro antes de qualquer token, não imprimem cabeçalho vazio.

5.3. processar e main

```

def processar(expressao):
    # --- desafio 2: converter para pós-fixa ---
    trace2, posfixa, erro = converter(expressao)
    imprimir_desafio2(trace2)

    if erro:
        print(erro)
        return

    print("posfixa:", posfixa)

    # --- desafio 1: avaliar a pós-fixa ---
    tokens = tokenizar_posfixa(posfixa)
    trace1, resultado, erro = avaliar(tokens)
    imprimir_desafio1(trace1)

    if erro:
        print(erro)
    else:
        print("resultado:", fmt(resultado))

def main():
    while True:
        expressao = input("Digite a expressão (vazio para sair): ")

        if expressao.strip() == "":
            break

        processar(expressao)
        print()

```

- **processar**: executa o fluxo completo para uma expressão. Se o *converter* devolve erro, imprime o trace até ali, imprime a mensagem e return (vai para a próxima expressão, sem encerrar o programa).
- Se a conversão deu certo, imprime "posfixa: ...", separa a pós-fixa em tokens, chama *avaliar*, imprime o trace e então o erro ou o "resultado:".
- **main**: laço que lê expressões até uma linha vazia. `print()` no final deixa uma linha em branco entre expressões.
- Nenhuma exceção escapa: todos os problemas previstos viram mensagens.

5.4. Exemplo completo de execução

```

token| pilha      | saida
(      | (              | -
3      | (              | 3
+      | ( +           | 3
4      | ( +           | 3 4
)      | -             | 3 4 +
*      | *             | 3 4 +
2      | *             | 3 4 + 2
(fim)| -             | 3 4 + 2 *
posfixa: 3 4 + 2 *
token| pilha
3      | 3
4      | 3 4
+      | 7
2      | 7 2
*      | 14
resultado: 14

```

6. Testes (resultados reais do código)

Os casos do enunciado, mais alguns extras (sem espaços, decimais, erros). Os valores desta tabela foram gerados rodando o próprio código.

Entrada	Pós-fixa	Resultado / erro
$3 + 4 * 2$	3 4 2 * +	11
$(3 + 4) * 2$	3 4 + 2 *	14
$1 + 2 * 3 - 4 / 2$	1 2 3 * + 4 2 / -	5
$3 + 4 * (2 - 1)$	3 4 2 1 - * +	7
$10 - 3 - 2$	10 3 - 2 -	5
$2 ^ 3 ^ 2$	2 3 2 ^ ^	512
$(2 ^ 3) ^ 2$	2 3 ^ 2 ^	64
$3+42*(7-1)$	3 4 2 7 1 - * +	255
$2.5 * 4$	2.5 4 *	10
$8 / (4 - 4)$	8 4 4 - /	erro: divisao por zero
$3 + 4)$	-	erro: parentese fechado sem abertura
$(3 + 4$	-	erro: parentese aberto sem fechamento
$3 \$ 4$	-	erro: caractere invalido

Casos da parte do Desafio 1 (pós-fixa direto):

Entrada pós-fixa	Resultado
3 4 +	7
3 4 2 * +	11
3 4 + 2 *	14
10 3 -	7
20 4 /	5
5 1 2 + 4 * + 3 -	14
3 + * 4	erro: operandos insuficientes
3 4 5 +	erro: expressao malformada
8 0 /	erro: divisao por zero

7. Decisões de implementação e limitações

- **Pilha encadeada, sem lista interna:** exigência do enunciado e coerente com o conteúdo da disciplina. Cada operação do contrato é $O(1)$.
- **Contador separado:** *tamanho()* em $O(1)$, e o *avaliar* usa *tamanho()* para validar operandos.
- **Tudo em float:** simplifica a divisão e os decimais. A função *fmt* cuida só da exibição.
- **Tabelas PRIORIDADE e DIREITA:** regras como dados; mudar uma regra é mudar uma tabela.
- **Funções puras:** *tokenizar*, *converter* e *avaliar* não imprimem; devolvem tuplas (trace, resultado, erro).
- **Trace por cópia:** *visualizar()* e *list(saida)* copiam os dados, senão todas as linhas do rastreamento mostrariam o estado final.
- **Erros como valor de retorno:** o programa nunca encerra por exceção; segue para a próxima expressão.

Limitações conhecidas (honestidade vale ponto na apresentação):

- **Números negativos** (por exemplo, $-3 + 4$) não são suportados: o $-$ é tratado só como operador binário. O enunciado não exige isso.
- **desempilhar() e topo()** não verificam pilha vazia; a segurança está nos chamadores.
- No Desafio 1, *float()* aceita palavras como "nan" e "inf"; o enunciado não pede tratamento desse caso.
- Expressão vazia em modo infixo gera pós-fixa vazia e cai em "expressao malformada" na avaliação.

8. Modificações que o professor pode pedir (com solução testada)

O enunciado avisa que **poderá ser solicitada uma pequena modificação no código durante a apresentação**. Estas são as mais prováveis; todas foram testadas no código de vocês.

8.1. Adicionar o operador % (módulo)

Três lugares mudam. **(a)** em desafio2.py, a tabela de prioridade e a lista de símbolos do tokenizar:

```
PRIORIDADE = {"+": 1, "-": 1, "*": 2, "/": 2, "%": 2, "^": 3}
...
elif caractere in "+-*/%^()":
```

(b) em desafio1.py, incluir "%" na tupla de operadores do *avaliar* e criar o ramo de cálculo (com proteção de divisão por zero):

```
if token in ("+", "-", "*", "/", "^", "%"):
    ...
    elif token == "%":
        if b == 0:
            return trace, None, "erro: divisao por zero"
        resultado = a % b
```

Teste: $7 \% 3 + 1$ gera $7 \% 3 + 1$ e resulta em 2.

8.2. Fazer o ^ associar à esquerda (ou outro operador à direita)

```
DIREITA = set() # nenhum à direita:  $2 \wedge 3 \wedge 2$  passa a dar 64
DIREITA = {"^", "-"} # (só como demonstração) - passaria a associar à direita
```

É por isso que as regras são dados: nenhuma linha da lógica precisa mudar.

8.3. Suportar números negativos (menos unário)

Ideia: um "-" é **unário** (sinal do número) quando está no início da expressão ou logo depois de outro operador ou de "(". Nesse caso, ele é lido como parte do número. Em desafio2.py, no *tokenizar*, **antes** do ramo de operadores:

```
elif caractere == "-" and (not tokens or tokens[-1] in ("+", "-", "*", "/", "^", "(")):
    fim = inicio + 1
    while fim < tamanho and (expressao[fim].isdigit() or expressao[fim] == "."):
        fim += 1
    numero = expressao[inicio:fim]
    try:
        float(numero)
    except ValueError:
        return None, "erro: caractere invalido"
    tokens.append(numero)
    inicio = fim
```

Testes feitos: $-3 + 4$ gera $-3 + 4$ (resultado 1); $2 * (-3)$ gera $2 -3 *$ (resultado -6); $5 - -2$ gera $5 -2 -$ (resultado 7); $3 - 4$ continua sendo subtração. Funciona porque o *converter* trata o token "-3" como número e o *avaliar* o lê com float.

8.4. Pequenas mudanças de impressão

- **Mostrar a pilha do topo para a base:** em *visualizar*, remover a linha `itens.reverse()`.

- **Mudar a largura das colunas:** alterar o 5 e o 11 em `{token:<5}` e `{texto(pilha):<11}`.
- **Mostrar casas decimais fixas:** trocar `f"{v:g}"` em *fmt* por `f"{v:.2f}"`.
- **Novo método na Pilha (ex.: limpar):** `def limpar(self): self.topo_no = None; self.contador = 0.`
- **Contar quantos operadores há na pilha:** percorrer a corrente com variável auxiliar, como no *visualizar*, ou usar `len(operadores.visualizar())`.

9. Perguntas prováveis e respostas

As perguntas são dirigidas **individualmente**. Cada integrante deve saber responder a todas, não só à sua parte.

Por que usar uma pilha neste problema?

Porque o operador que chega pode não poder ser aplicado ainda, então precisa ser guardado, e a ordem de retomada é sempre "o mais recente primeiro". Isso é o contrato LIFO da pilha. Não é conveniência: é a estrutura do problema.

O que a pilha guarda em cada desafio?

No Desafio 1, os números (operandos) que aguardam um operador. No Desafio 2, os operadores e parênteses que ainda não puderam ir para a saída.

Por que no avaliar o primeiro desempilhado é o b e não o a?

Porque o topo da pilha é o último número empilhado, que é o operando da direita. Se o código fizesse o contrário, $5 \cdot 2$ - daria $2 \cdot 5$.

Por que o $\wedge$ usa ">" e os outros usam ">="?

Por associatividade. Com ">=", operadores iguais saem antes, o que resolve da esquerda para a direita ($10 - 3 - 2 = 5$). Com ">" o operador igual permanece na pilha, então o último é resolvido primeiro, o que é a direita para a esquerda ($2 \wedge 3 \wedge 2 = 512$).

Por que o código testa `topo() != "("` no laço?

O parêntese de abertura é uma barreira. Operadores de dentro do parêntese não podem ser comparados com o que está fora. Só o ramo do ")" remove o "(".

Por que o visualizar copia os valores em vez de devolver a pilha?

Para não destruir a pilha e não expor sua estrutura interna. Ele usa uma variável auxiliar para percorrer, sem mexer no `topo_no`, e devolve uma cópia em lista.

Por que o trace usa `list(saída)`?

Sem a cópia, todas as linhas do trace apontariam para a mesma lista e mostrariam o resultado final, porque a lista é um objeto mutável compartilhado.

Por que não dá para tokenizar com `split()`?

Porque a entrada pode vir sem espaços ($3+42*(7-1)$). É preciso reconhecer que dígitos e ponto continuam o número e que qualquer outro símbolo começa um token novo.

Qual a complexidade do algoritmo?

Cada token é empilhado e desempilhado no máximo uma vez, então a conversão e a avaliação são $O(n)$, com operações de pilha $O(1)$. O visualizar é $O(n)$ por chamada e só serve para imprimir o rastreamento.

Por que o contador existe?

Para `tamanho()` ser $O(1)$. Sem ele seria preciso percorrer todos os nós. O `avaliar` usa `tamanho()` para validar se há 2 operandos.

Por que usar float para tudo?

Simplifica a divisão ($20 / 8 = 2.5$) e aceita decimais. A função `fmt` remove o ".0" só na impressão.

O que acontece em `3 4 5 +` e em `3 + * 4`?

No primeiro, o `+` consome 4 e 5 e sobram 2 números na pilha: "expressao malformada". No segundo, o `+` chega com apenas 1 número: "operandos insuficientes".

Qual a diferença entre infixa e pós-fixa?

Na infixa o operador fica entre os operandos e precisa de precedência e parênteses. Na pós-fixa o operador vem depois dos operandos e a ordem dos símbolos já elimina a ambiguidade.

Por que `desempilhar()` não verifica pilha vazia?

Decisão de projeto: manter a pilha simples e deixar a validação para quem usa, que já sabe o contexto (o `avaliar` checa tamanho, o `converter` checa vazia). A limitação está documentada.

Por que há duas funções tokenizar?

São problemas diferentes: a pós-fixa já vem separada por espaço (`split` basta); a notação usual pode vir sem espaços e exige leitura caractere a caractere. Na `main`, uma é importada com apelido para evitar conflito.

10. Roteiro sugerido para a apresentação

Sugestão de divisão em três partes (ajustem entre vocês; o importante é todos dominarem tudo).
Tempo estimado: 10 a 15 minutos, mais perguntas.

Parte	Conteúdo	Seções deste material
1. Contexto e Pilha	O problema, por que uma pilha, a classe No e Pilha, complexidade e o método visualizar	1 e 2
2. Desafio 1	Pós-fixa, o algoritmo de avaliação, a ordem b depois a, erros, demonstração ao vivo de $3\ 4\ 2\ *\ +$	3
3. Desafio 2 e main	Tokenização, PRIORIDADE e DIREITA, a regra $\geq$ e $>$, demonstração de $2\ ^\wedge\ 3\ ^\wedge\ 2$ e $10 - 3 - 2$, a main encadeando tudo	4 e 5

Demonstração ao vivo recomendada (nesta ordem):

- $3\ +\ 4\ *\ 2$: mostra precedência (pilha "+ *").
- $(3\ +\ 4)\ *\ 2$: mostra o parêntese como muro.
- $10\ -\ 3\ -\ 2$ e $2\ ^\wedge\ 3\ ^\wedge\ 2$: mostra a associatividade e a diferença entre $\geq$ e $>$.
- $3+42*(7-1)$: mostra a tokenização sem espaços.
- $3\ +\ 4$) e $8\ /\ (4\ -\ 4)$: mostra o tratamento de erros sem exceção.

11. Declaração de uso de ferramentas de IA

Conforme o enunciado, o uso de IA como ferramenta de estudo deve ser declarado no final do registro.md. A equipe já declarou o uso do Claude no registro: explicação de conceitos (classes, *self*, nós encadeados, pilha, conversão infixa para pós-fixa), sugestão de nomes de variáveis, revisão frente ao enunciado e apoio na redação. **Este PDF é um material de estudo e apresentação gerado com apoio do Claude a partir do código e do registro da equipe**; se for usado, convém acrescentar essa finalidade à declaração do registro.md. O código deve ser conhecido e explicável por cada integrante.